

빅데이터분석 기본 개념문제 풀이

㉠ Q1. TF-IDF에서 TF와 IDF 각각이 어떤 현상을 보정하기 위해 존재하는지 설명하시오.

㉠ 상세 풀이 >

TF-IDF(Term Frequency-Inverse Document Frequency)는 단어의 중요도를 평가하는 통계적 가중치입니다. TF와 IDF는 각각 다음 현상을 보정하기 위해 존재합니다.

- **TF (Term Frequency, 단어 빈도):**
 - 보정 대상: 문서의 길이에 따른 단어 빈도의 왜곡.
 - 설명: TF는 특정 단어가 한 문서 내에 얼마나 자주 등장하는지를 나타냅니다. 하지만 문서의 길이가 길면 특정 단어의 등장 횟수가 자연스럽게 높아지는 문제가 있습니다. 이를 보정하기 위해 TF는 단순히 단어의 출현 횟수를 세는 것(Raw TF)을 넘어, 문서의 총 단어 수로 나누어 정규화(Normalized TF)하는 방식을 사용합니다. 이를 통해 문서 길이에 상관없이 단어의 상대적인 중요도를 공정하게 비교할 수 있습니다.
- **IDF (Inverse Document Frequency, 역문서 빈도):**
 - 보정 대상: 모든 문서에 흔하게 등장하는 단어(불용어, stopword)의 중요도 과대평가.
 - 설명: 'the', 'a', 'is'와 같이 거의 모든 문서에 등장하는 단어들은 단어 빈도(TF)만 높을 뿐, 문서의 주제를 구별하는 데는 거의 기여하지 못합니다. IDF는 이러한 단어들의 중요도를 낮추기 위해 존재합니다. 특정 단어가 여러 문서에 걸쳐 얼마나 흔하게 등장하는지를 측정하여, 그 역수를 취합니다. 즉, 많은 문서에 등장할수록 IDF 값은 낮아지고, 소수의 특정 문서에만 집중적으로 등장할수록 IDF 값은 높아집니다. 이를 통해 문서의 주제를 잘 나타내는 핵심 단어에 더 높은 가중치를 부여할 수 있습니다.

㉠ Q2. IDF에서 로그를 사용하는 이유?

㉠ 상세 풀이 >

IDF(Inverse Document Frequency) 계산에 로그(log)를 사용하는 주된 이유는 데이터 값의 규모를 조정(**scaling**)하고, 단어의 희귀성에 따른 가중치 변화를 완만하게 만들기 위함입니다.

$$\text{IDF}(t, D) = \log \left(\frac{N}{|\{d \in D : t \in d\}|} \right)$$

여기서 N 은 총 문서의 수, 분모는 단어 t 를 포함하는 문서의 수입니다.

로그를 사용하는 구체적인 이유:

1. 값의 폭발적 증가 방지 (Dampening Effect):

- 총 문서 수가 100만 개라고 가정해 봅시다. 단어 A가 10개 문서에, 단어 B가 1개 문서에만 등장한다면, 로그를 사용하지 않을 경우 IDF 값은 각각 100,000과 1,000,000이 됩니다. 이처럼 값의 차이가 너무 커지면, 희귀 단어 하나가 다른 단어들의 중요도를 압도해버리는 문제가 발생할 수 있습니다.
- 로그를 사용하면 이 값들이 $\log(100,000) \approx 11.5$, $\log(1,000,000) \approx 13.8$ 과 같이 훨씬 작은 규모로 줄어듭니다. 이를 통해 IDF 값이 너무 급격하게 커지는 것을 막고, 전체적인 가중치 분포를 안정적으로 만듭니다.

2. 단어의 정보량 관점:

- 정보 이론에서 어떤 사건의 정보량은 해당 사건이 발생할 확률에 로그를 취한 값으로 표현됩니다 ($-\log P(x)$). IDF는 이와 유사한 개념으로 해석될 수 있습니다.
- 매우 희귀한 단어(등장 확률이 낮은 단어)가 나타났을 때, 그 단어는 더 많은 정보량을 가집니다. 로그는 이러한 정보량의 증가를 선형적(linear)이 아닌, 점차 완만해지는 곡선으로 표현하여 단어의 희귀성이 정보량에 기여하는 방식을 더 현실적으로 모델링합니다.

3. 계산의 안정성:

- IDF 값들을 다른 값들과 곱하거나 더할 때, 로그를 취한 값들은 수치적으로 더 다루기 쉽고 안정적입니다.

② Q3. Power Law가 문서 빈도 분포에서 어떻게 나타나는지 기술하시오.

③ 상세 풀이 >

Power Law (멱법칙)는 문서 빈도 분포에서 "소수의 단어가 매우 자주 등장하고, 대다수의 단어는 매우 드물게 등장하는" 현상으로 나타납니다. 이를 지프의 법칙(**Zipf's Law**)이라고도 하며, 다음과 같은 특징을 보입니다.

1. 소수의 '헤비 유저' 단어:

- 문서 집합 전체에서 가장 자주 등장하는 몇몇 단어(예: 'the', 'a', 'is' 등 불용어)가 전체 단어 출현 빈도의 상당 부분을 차지합니다.
- 예를 들어, 상위 10개 단어가 전체 단어 발생 횟수의 20~30%를 점유할 수 있습니다.

2. 긴 꼬리(Long Tail)를 가진 대다수의 희귀 단어:

- 그래프의 오른쪽으로 갈수록, 즉 단어의 등장 빈도 순위가 낮아질수록, 해당 빈도를 가진 단어의 수는 기하급수적으로 많아집니다.
- 이러한 단어들은 각각의 등장 빈도는 매우 낮지만(예: 1~2번 등장), 그 종류가 매우 많아 전체 단어 집합의 대부분을 형성합니다. 이 부분을 "롱테일(Long Tail)"이라고 부릅니다.

시각적 표현:

문서 빈도 분포를 그래프로 그리면 Power Law 현상을 명확하게 확인할 수 있습니다.

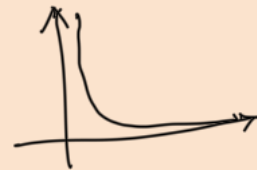
- **x축:** 단어의 등장 빈도 순위 (Rank)
- **y축:** 단어의 등장 빈도 (Frequency)

이 그래프를 일반적인 선형 스케일(linear scale)로 그리면, 왼쪽 상단에 매우 가파른 절벽이 형성되고 오른쪽으로 갈수록 축에 거의 붙어 길게 이어지는 형태가 나타납니다.

하지만 **x축**과 **y축** 모두에 로그 스케일(log-log scale)을 적용하면, 이 그래프는 거의 직선에 가까운 형태를 띠게 됩니다. 이는 Power Law 분포의 전형적인 특징입니다.

$$\text{frequency} \propto \frac{1}{\text{rank}^a}$$

즉, 단어의 등장 빈도는 해당 단어의 순위에 반비례하는 멱법칙 관계를 가집니다.



② Q4. 스트리밍 데이터 처리(stream processing)가 필요한 이유를 두 가지 쓰시오.

③ 상세 풀이 >

스트리밍 데이터 처리는 데이터가 생성되는 즉시 실시간으로 처리하는 방식으로, 다음과 같은 두 가지 핵심적인 이유 때문에 필요합니다.

1. 실시간 의사결정 및 즉각적인 대응의 필요성:

- 많은 현대 비즈니스 환경에서는 데이터를 모아서 나중에 일괄 처리(batch processing)하는 것으로는 충분하지 않습니다. 사기 탐지, 주가 예측, 실시간 추천, 공장 설비 모니터링 등에서는 수 초 또는 밀리초 단위의 빠른 분석과 즉각적인 대응이 매우 중요합니다.
- 예를 들어, 신용카드 부정 사용이 감지되면 즉시 거래를 차단해야 하고, 웹사이트 사용자의 행동에 따라 즉각적으로 맞춤형 광고를 제공해야 합니다. 스트리밍 처리는 이처럼 시간적 가치가 매우 높은 데이터를 지체 없이 처리하여 비즈니스 기회를 포착하거나 위기를 방지할 수 있게 해줍니다.

2. 대용량 데이터의 효율적인 처리 및 I/O 바운디드 문제 해결:

- IoT 센서, 소셜 미디어, 로그 데이터 등 오늘날의 데이터 소스는 **저장하기 어려울 정도로 방대한 양의 데이터를 끊임없이 생성**합니다. 이 모든 데이터를 디스크에 저장한 후 처리하는 **배치(batch) 방식**은 잦은 디스크 I/O를 발생시켜 **I/O 바운디드(I/O-bound)** 문제가 되기 쉬우며, 막대한 저장 비용과 처리 시간을 유발합니다.
- 스트리밍 데이터 처리는 데이터를 저장하지 않고 **인메모리(in-memory) 기반으로 흘러가는 대로(on-the-fly) 처리**하므로, 저장 공간에 대한 부담이 적고 시스템 자원, 특히 I/O 자원을 훨씬 효율적으로 사용할 수 있습니다. 전체 데이터를 보관할 필요 없이 필요한 정보만 추출하거나 요약하여 저장할 수 있어 대용량 데이터를 경제적으로 다룰 수 있습니다.

Q5. Poll, push, pull 방식의 차이점

상세 풀이 >

Poll, Push, Pull은 분산 시스템이나 데이터 처리 환경에서 데이터를 주고받는 방식의 차이를 나타냅니다. 주체(누가 요청하고 누가 응답하는지)를 기준으로 구분할 수 있습니다.

- Pull (끌어오기 방식):**
 - 주체: **소비자(Consumer)**가 데이터가 필요할 때 **생산자(Producer)**에게 능동적으로 요청하여 데이터를 가져오는 방식입니다.
 - 특징: 소비자가 자신의 처리 능력에 맞춰 데이터 흐름을 조절할 수 있습니다. 하지만 데이터가 언제 생성될지 모르기 때문에, 소비자는 주기적으로 생산자에게 "새로운 데이터가 있나요?"라고 물어봐야 합니다(polling). 이 과정에서 불필요한 요청이 발생하거나 데이터 수신이 지연될 수 있습니다.
 - 예시: 웹 브라우저가 F5 키를 눌러 웹 서버로부터 최신 페이지를 가져오는 것, Kafka 컨슈머가 브로커로부터 메시지를 가져오는 것.
- Push (밀어주기 방식):**
 - 주체: **생산자(Producer)**가 새로운 데이터가 생겼을 때 **소비자(Consumer)**에게 능동적으로 데이터를 보내주는 방식입니다.
 - 특징: 데이터가 생성되는 즉시 소비자에게 전달되므로 실시간 성이 매우 높습니다. 하지만 소비자의 처리 능력을 고려하지 않고 데이터를 계속 보내면 소비자가 과부하에 걸릴 수 있습니다.
 - 예시: 웹 서버가 웹소켓(WebSocket)을 통해 클라이언트에게 실시간 알림을 보내는 것, 모바일 앱 푸시 알림.
- Poll (폴링):**
 - 개념: Poll은 Pull 방식을 구현하기 위한 **구체적인 행위를 지칭**하는 경우가 많습니다. 즉, 소비자가 주기적으로 생산자의 상태를 **"확인(checking)"**하는 동작 자체를 의미합니다.
 - 특징: Pull과 거의 동일한 의미로 사용되지만, 특히 '주기적인 확인'이라는 행위에 초점을 맞춥니다. "Pull 하기 위해 Poll 한다"고 이해할 수 있습니다.
 - 문제점: 실시간성이 떨어지고, 데이터 변경이 없을 때도 불필요한 네트워크 트래픽을 유발합니다.

Q6. 클러스터 컴퓨팅에서 발생하는 세 가지 핵심 문제(Node failure, Network bottleneck, Programming complexity)를 설명하시오.

상세 풀이 >

클러스터 컴퓨팅은 여러 대의 컴퓨터(노드)를 연결하여 하나의 고성능 컴퓨터처럼 사용하는 기술이지만, 이로 인해 단일 컴퓨터 환경에서는 없었던 다음과 같은 세 가지 핵심적인 문제가 발생합니다.

1. 노드 장애 (Node Failure):

- 문제: 클러스터는 수십, 수백, 수천 개의 노드로 구성됩니다. 노드의 수가 많아질수록, 그중 일부가 하드웨어 고장, 소프트웨어 오류 등으로 인해 **장애를 일으킬 확률이 매우 높아**집니다. 단일 컴퓨터라면 시스템 전체가 멈추겠지만, 클러스터에서는 일부 노드의 장애가 전체 시스템의 중단으로 이어지지 않도록 만들어야 합니다.

- **해결:** MapReduce, Spark와 같은 분산 처리 프레임워크는 내고장성(Fault Tolerance)을 통해 이 문제를 해결합니다. 데이터를 여러 노드에 복제하여 저장(예: HDFS)하고, 특정 노드에서 작업이 실패하면 다른 정상 노드에서 해당 작업을 자동으로 재실행하는 메커니즘을 갖추고 있습니다.

2. 네트워크 병목 (Network Bottleneck):

- **문제:** 클러스터의 노드들은 작업을 수행하기 위해 네트워크를 통해 데이터를 주고받습니다. 특히 MapReduce나 Spark의 셔플(Shuffle) 단계에서는 대규모 데이터가 노드 간에 이동하는데, 이때 네트워크 대역폭이 데이터 이동 속도를 따라가지 못하면 네트워크가 전체 시스템의 성능을 저하 시키는 병목 지점이 됩니다.
- **해결:** 데이터 지역성(Data Locality)을 활용하여, 계산을 데이터가 저장된 노드에서 최대한 직접 수행하도록 스케줄링하여 네트워크 트래픽을 최소화합니다. 또한, 데이터를 전송하기 전에 압축하거나, 셔플 과정의 데이터 이동을 최적화하는 등의 기법을 사용합니다.

3. 프로그래밍 복잡성 (Programming Complexity):

- **문제:** 여러 노드에서 작업을 병렬로 실행하고, 데이터를 분산시키며, 노드 간 통신과 동기화를 관리하고, 장애에 대비하는 모든 과정을 프로그래머가 직접 코드로 구현하는 것은 매우 복잡하고 어렵습니다.
- **해결:** Hadoop MapReduce, Apache Spark와 같은 분산 컴퓨팅 프레임워크가 이러한 복잡성을 추상화하여 프로그래머에게 간단한 API(예: `map`, `reduce`, `filter`)를 제공합니다. 프로그래머는 비즈니스 로직에만 집중하면 되고, 프레임워크가 내부적으로 데이터 분산, 병렬 처리, 장애 복구 등을 자동으로 처리해 줍니다.

Q7. Spark의 WordCount 구현은 MapReduce와 어떤 두 가지 핵심 차이가 있는가? (RDD 특성 및 Lazy Evaluation 관점에서 답하시오.)

상세 풀이 >

Spark의 WordCount는 Hadoop MapReduce의 WordCount와 결과적으로는 같은 작업을 수행하지만, 내부 동작 방식에서 다음과 같은 두 가지 핵심적인 차이점을 보입니다.

1. 데이터 처리 방식: 메모리 기반 처리 vs. 디스크 기반 처리 (RDD 특성)

- **MapReduce:** MapReduce는 각 처리 단계(Map, Reduce)의 중간 결과를 반드시 디스크(HDFS)에 기록합니다. 예를 들어, Map 작업의 결과는 디스크에 저장되고, Reduce 작업은 이 디스크에 저장된 데이터를 다시 읽어서 처리합니다. 이 과정에서 잦은 디스크 I/O가 발생하여 성능 저하의 주요 원인이 됩니다.
- **Spark:** Spark는 RDD(Resilient Distributed Dataset)라는 추상화된 데이터 모델을 사용합니다. RDD는 기본적으로 메모리에 데이터를 저장하고 처리합니다. WordCount 과정에서 `map`, `reduceByKey` 등의 연산 중간 결과는 디스크가 아닌 메모리상에서 파이프라이닝(pipelining) 방식으로 전달됩니다. 이 In-Memory Computing 덕분에 디스크 I/O 병목이 크게 줄어들어 MapReduce보다 훨씬 빠른 성능을 보입니다.

2. 실행 시점: 지연 평가 vs. 즉시 평가 (Lazy Evaluation 관점)

- **MapReduce:** MapReduce 작업은 제출되는 즉시 각 단계가 실행 계획에 따라 실행됩니다.
- **Spark:** Spark의 RDD 연산은 지연 평가(Lazy Evaluation) 방식으로 동작합니다. `sc.textFile()`, `flatMap()`, `map()` 과 같은 변환(Transformation) 연산은 호출 즉시 실행되지 않고, 어떤 연산을 수행해야 하는지에 대한 계획(DAG, Directed Acyclic Graph)만 생성합니다. 실제 계산은 `count()`, `collect()`, `saveAsTextFile()` 과 같은 액션(Action) 연산이 호출되었을 때 비로소 시작됩니다. 이 지연 평가 덕분에 Spark는 전체 작업 흐름을 파악하고, 여러 연산을 묶어서 처리하거나 데이터 소스에서 필요한 부분만 읽는 등 전역적인 최적화를 수행할 수 있어 더 효율적입니다.

Q8. 간단하게 설명하세요

- (1) Mean Imputation
- (2) Model-based Imputation
- (3) PCA의 첫 번째 주성분(PC1)의 의미
- (4) 지니 불순도
- (5) 엔트로피

- (6) WoE(Weight of Evidence)가 필요한 이유
- (7) 단조성(Monotonicity)이 변수 선택에서 중요한 이유

② 상세 풀이 >

(1) Mean Imputation (평균 대체법):

데이터의 결측치(missing value)를 해당 변수의 **평균값**으로 채우는 가장 간단한 결측치 처리 방법입니다. 계산이 매우 빠르고 쉽지만, 데이터의 원래 분산을 왜곡시키고 변수 간의 상관관계를 약화시킬 수 있는 단점이 있습니다.

(2) Model-based Imputation (모델 기반 대체법):

결측치를 채우기 위해 머신러닝 모델을 사용하는 방법입니다. 결측치가 있는 변수를 타겟 변수로, 나머지 변수들을 예측 변수로 사용하여 회귀(regression)나 분류(classification) 모델을 학습시킨 후, 그 모델의 예측값으로 결측치를 대체합니다. 평균 대체법보다 훨씬 정교하지만 계산 비용이 높습니다.

(3) PCA의 첫 번째 주성분(PC1)의 의미:

첫 번째 주성분(Principal Component 1)은 **원본 데이터의 분산(variance)**을 가장 잘 설명하는 방향 벡터입니다. 즉, 데이터를 PC1이라는 하나의 축에 투영(projection)했을 때, 데이터 포인트들이 가장 넓게 퍼져 분산이 최대가 되는 방향을 의미하며, 데이터의 가장 중요한 정보의 축이라고 할 수 있습니다.

(4) 지니 불순도 (Gini Impurity):

의사결정나무(Decision Tree)에서 사용되는 지표로, 특정 노드에 속한 데이터들이 얼마나 다른 클래스들로 섞여 있는지를 측정합니다. 지니 불순도가 0이면 해당 노드의 모든 데이터가 단 하나의 클래스로 구성된 순수한 상태이고, 값이 높을수록 여러 클래스가 불순하게 섞여 있음을 의미합니다. 의사결정나무는 이 지니 불순도를 가장 많이 낮추는 방향으로 데이터를 분기합니다.

(5) 엔트로피 (Entropy):

지니 불순도와 유사하게 데이터의 불확실성(uncertainty) 또는 무질서도를 측정하는 지표입니다. 엔트로피가 높으면 데이터가 여러 클래스로 균등하게 섞여 있어 예측이 어려운 상태를 의미하고, 0이면 모든 데이터가 하나의 클래스에 속해 불확실성이 전혀 없는 상태를 의미합니다. 정보 이득(Information Gain)은 이 엔트로피를 얼마나 줄였는지를 측정합니다.

(6) WoE(Weight of Evidence)가 필요한 이유:

WoE는 특정 변수의 범주가 타겟 변수(예: 부도/정상)를 얼마나 잘 예측하는지를 나타내는 가중치입니다. 로지스틱 회귀와 같은 모델에서 WoE를 사용하는 이유는 다음과 같습니다.

1. 범주형 변수를 모델이 이해할 수 있는 연속적인 숫자형 변수로 변환합니다.
2. WoE 값 자체가 해당 범주의 예측력을 나타내므로 변수의 중요도를 직관적으로 파악할 수 있습니다.
3. 모든 변수를 동일한 스케일로 변환하여 모델의 안정성을 높입니다.

(7) 단조성(Monotonicity)이 변수 선택에서 중요한 이유:

단조성(한 변수가 증가할 때 다른 변수도 꾸준히 증가하거나 감소하는 경향)은 변수 선택에서 중요합니다. 왜냐하면 로지스틱 회귀와 같은 많은 모델은 **입력 변수와 예측 확률 간에 단조적인 관계가 있다고 가정하기** 때문입니다. 만약 변수가 단조성을 만족하지 않는다면(예: 나이가 증가하다가 특정 시점부터는 위험도가 다시 감소), 모델이 관계를 제대로 학습하기 어려우며, 이는 모델의 예측 성능과 안정성을 저해합니다. 따라서 변수 선택 시 단조성을 보이는 변수를 우선적으로 고려하는 것이 좋습니다.

② Q9. 간단하게 설명하세요

- (1) map vs flatMap
- (2) persistence vs cache
- (3) Broadcast Variable의 필요성
- (4) Accumulator가 사용 가능한 상황
- (5) Lazy Evaluation이 성능을 높이는 이유

② 상세 풀이 >

(1) map vs flatMap:

- **map:** RDD의 각 요소(element)에 함수를 적용하여 일대일(one-to-one)로 새로운 RDD를 생성합니다. 입력 요소 하나당 출력 요소 하나가 나옵니다. (예: `[1, 2, 3] -> [2, 4, 6]`)
- **flatMap:** RDD의 각 요소에 함수를 적용한 후, 그 결과(주로 리스트나 배열)를 "펼쳐서(flatten)" 하나의 RDD로 만듭니다. 일대다(one-to-many) 변환이 가능합니다. (예: `["hello world", "hi"] -> ["hello", "world", "hi"]`)

(2) persistence vs cache:

`cache()`와 `persist()`는 모두 RDD를 재사용하기 위해 저장하는 연산이지만, 저장 방식에 차이가 있습니다.

- **cache():** RDD를 오직 메모리(MEMORY_ONLY)에만 저장하는 가장 단순한 형태의 영속화(persistence)입니다.
- **persist():** RDD를 메모리, 디스크, 또는 둘 다에 저장하는 등 다양한 저장 레벨(Storage Level)을 직접 지정할 수 있습니다. 예를 들어, `persist(StorageLevel.MEMORY_AND_DISK)`는 메모리가 부족하면 디스크를 함께 사용하도록 설정할 수 있습니다. 즉, `cache()`는 `persist()`의 특별한 경우(기본값)라고 할 수 있습니다.

(3) Broadcast Variable의 필요성:

Broadcast Variable(브로드캐스트 변수)은 상대적으로 크기가 큰 읽기 전용(read-only) 데이터를 클러스터의 모든 노드에 효율적으로 전송하기 위해 필요합니다. 일반 변수를 사용하면 해당 변수가 필요한 모든 태스크(Task)마다 드라이버에서 Executor로 반복적으로 전송되어 심각한 네트워크 병목을 유발합니다. 브로드캐스트 변수를 사용하면 각 Executor에 단 한 번만 데이터를 복사해 놓고, 해당 Executor 내의 모든 태스크가 이를 공유해서 사용하므로 네트워크 비용을 크게 절감할 수 있습니다.

(4) Accumulator가 사용 가능한 상황:

Accumulator(누산기)는 클러스터 전체에서 분산 환경의 각 태스크들이 값을 안전하게 누적(aggregate)할 수 있는 공유 변수가 필요할 때 사용됩니다. 일반 변수는 각 Executor에서 변경되더라도 드라이버에는 반영되지 않지만, Accumulator를 사용하면 모든 노드에서 수행된 연산(예: 덧셈, 카운팅) 결과를 드라이버에서 최종적으로 합산할 수 있습니다. 디버깅을 위한 카운터나 전체 처리 건수, 오류 수 등을 집계하는 상황에서 유용합니다.

(5) Lazy Evaluation이 성능을 높이는 이유:

Lazy Evaluation(지연 평가)은 Spark에서 액션 연산이 호출되기 전까지 변환 연산을 실행하지 않는 원리입니다. 이 방식이 성능을 높이는 이유는 다음과 같습니다.

1. **전역 최적화 (Global Optimization):** Spark는 전체 작업 흐름(DAG)을 미리 파악하고, 여러 연산을 하나로 합치거나(Pipelining), 불필요한 데이터 로딩을 생략하는 등 전체 프로세스를 최적화한 후 실행할 수 있습니다.
2. **데이터 처리량 감소:** 예를 들어, `filter` 연산 후 `take(10)`을 수행할 때, 전체 데이터를 필터링하지 않고 조건에 맞는 10개의 데이터만 찾으면 즉시 작업을 멈출 수 있어 불필요한 계산을 줄입니다.

Q10. MLflow의 Tracking, Projects, Models, Registry 네 가지 구성요소를 1-2줄로 요약하시오.

☞ 상세 풀이 >

- **Tracking:** 머신러닝 실험의 파라미터, 코드 버전, 평가지표(metrics), 산출물(artifacts) 등을 기록하고 조회하는 컴포넌트입니다. 이를 통해 실험 과정을 추적하고 결과를 재현할 수 있습니다.
- **Projects:** 코드, 종속성, 실행 환경 등을 표준화된 형식(MLproject 파일)으로 패키징하여, 어떤 플랫폼에서든 동일한 코드를 재현하고 실행할 수 있도록 보장하는 컴포넌트입니다.
- **Models:** 다양한 머신러닝 라이브러리로 학습된 모델을 표준화된 형식(flavor)으로 저장하고 배포하는 컴포넌트입니다. 이를 통해 특정 라이브러리에 종속되지 않고 모델을 서빙할 수 있습니다.
- **Registry:** 학습된 MLflow 모델의 전체 생명주기(개발, 스테이징, 프로덕션 등)를 관리하는 중앙 저장소입니다. 모델의 버전을 관리하고, 상태를 추적하며, 협업을 용이하게 합니다.

🕒 Q11. 파레토 분석

- (1) “해 x_1 이 해 x_2 를 지배한다”의 조건을 두 가지로 쓰시오.
- (2) 파레토 최적 집합과 파레토 프론트의 차이를 설명하시오.

🕒 상세 풀이 >

(1) “해 x_1 이 해 x_2 를 지배한다(dominates)”의 조건 (다중 목표 최소화 문제 기준):

해(solution) x_1 이 다른 해 x_2 를 지배하기 위해서는 다음 두 가지 조건을 모두 만족해야 합니다.

1. 모든 목표(objective)에서 x_1 이 x_2 보다 나쁘지 않아야 한다: 즉, 모든 목표 함수 값에 대해, x_1 의 결과값이 x_2 의 결과값보다 작거나 같아야 합니다.
2. 적어도 하나의 목표에서 x_1 이 x_2 보다 명백히 더 우수해야 한다: 즉, 여러 목표 함수 값 중 적어도 하나 이상에서 x_1 의 결과값이 x_2 의 결과값보다 명백히 작아야 합니다.

(2) 파레토 최적 집합과 파레토 프론트의 차이:

- 파레토 최적 집합 (Pareto Optimal Set): 결정 변수 공간(decision variable space)에서 다른 어떤 해에도 지배당하지 않는 모든 해(solution)들의 집합입니다. 즉, 입력값(파라미터)들의 모음입니다.
- 파레토 프론트 (Pareto Front): 목표 공간(objective space)에서 파레토 최적 집합에 속한 해들의 목표 함수 값들로 이루어진 곡선 또는 표면입니다. 즉, 파레토 최적 해들이 만들어내는 성능 결과(출력값)들의 시각적인 형태입니다.